

CSE 719 — DSM & Consistency Models Solutions

Lecture-09 · Exam: Wed 1 Jul 2026 · Dr. Atiqur Rahman

Contents

1	Reference: DSM & Consistency Quick Facts	2
2	2021 Q8b / 2022 Q8b — Define DSM; Implement over Message-Passing (3–3.5 marks)	2
3	2024 Q4a — Message Passing vs DSM Approaches (1.5 marks)	3
4	2020 Q6b / 2021 Q3c / 2022 Q4c / 2024 Q4b — False Sharing (1.5–3 marks each)	4
5	2022 Q8c — Invalidate vs Update Protocol; Which is Preferable? (3 marks)	5
6	2020 Q4c — Why Simple LRU Fails as DSM Block Replacement Policy (2.25 marks)	5
7	2020 Q6c — NRNMB / NRMB / RMB / RNMB Replication Strategies (4 marks)	6
8	2021 Q3d / 2022 Q4d — Consistency: Definition + Types in DSM (2–3 marks)	7
9	2020 Q4a — Causal Consistency + Application (3 marks)	8
10	2020 Q4b — Weak vs Release Consistency; Which for DSM? (3.5 marks)	9
11	2021 Q2a / 2021 Q2b — Strong vs Weak for Specific Scenarios (3 + 1.75 marks)	10
12	2021 Q2c — Quorum Read/Write Constraints: 9 Replicas (4 marks)	11
13	2024 Q2b — Linearizability + Methods to Ensure Serializability (1.5 marks)	12
14	Summary Table	13

Reference: DSM & Consistency Quick Facts

DSM in one sentence: An abstraction in which processes on different machines virtually share memory pages, programmed as if on a shared-memory multiprocessor, implemented over a message-passing network.

Consistency model ladder (weakest → strongest):

Model	Guarantee	Cost
Eventual	Writes propagate eventually; no ordering	Lowest
PRAM/FIFO	Writes by one process seen in order by all	Low
Causal	Causally related writes seen in causal order everywhere	Medium
Release	Flushes shared state on release; consistent at lock boundaries	Medium
Weak	Consistent only at synchronisation points	Medium
Sequential	All see same total order of operations	High
Linearizability	Sequential + real-time: each op appears instantaneous	Highest

Invalidate vs Update in one line:

- **Invalidate:** on write → multicast *invalidate* to all R-copy holders → only owner has copy (W state).
- **Update:** on write → multicast *new value* to all holders → all update in place.

Quorum rules for N replicas: $R + W > N$ and $W > N/2$.

2021 Q8b / 2022 Q8b — Define DSM; Implement over Message-Passing (3–3.5 marks)

Question: Define DSM. Discuss/implement DSM over a message-passing network with suitable illustration.

Definition

Distributed Shared Memory (DSM) is a software abstraction that allows processes running on separate machines in a message-passing network to access a common virtual address space as if it were physically shared memory. Each process sees the same global set of memory pages; reads and writes are intercepted by the DSM layer and translated into network messages when necessary.

Advantages:

- Allows reuse of shared-memory programs without rewriting for message passing.
- Simpler programming model (shared variables) compared to explicit send/receive.

Implementation over Message-Passing (Invalidate Protocol)

Key mechanism:

1. Each process maintains a **local cache** of recently accessed pages.
2. On access, process checks cache: **page hit** → serve from cache (no network). **Page miss** → **page fault** (kernel trap) → DSM software activated.

3. DSM software locates the page owner via **multicast**, fetches the page, caches it, and resumes the process.

Page states and ownership:

- **Owner** = process with the latest version of a page.
- **R state**: owner has R copy; other processes *may* also hold R copies; no W copy exists.
- **W state**: *only* the owner has a copy.

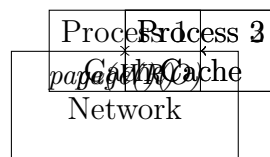
Read operation (Process 1 wants to read a page):

P1's state	Others' state	Action
Has page (R or W, owner or not)	Any	Read from cache. No messages sent.
No page	Others hold R copies	Multicast: ask for R copy. Mark page R.
No page	Another holds W copy	Ask owner to degrade to R. Get page. Mark page R.

Write operation (Process 1 wants to write a page):

P1's state	Others' state	Action
Owner, W state	No other copies	Write to cache. No messages sent.
Owner, R state	Others may have R	Multicast invalidate all R copies. Mark W. Write.
Has R, not owner	Others may have R	Multicast invalidate all R copies. Mark W. Become owner.
No page	Any	Multicast invalidate all copies. Fetch page. Mark W. Become owner.

Schematic (DSM over message-passing):



On a write by P1: P1 multicasts INVALIDATE to P3 (and all other R-copy holders). P3 discards its cached copy. P1 marks its page W and proceeds.

2024 Q4a — Message Passing vs DSM Approaches (1.5 marks)

Question: Compare the message passing and DSM approaches to distributed computation.

	Message Passing	DSM
Programming model	Explicit <code>send/receive</code> ; programmer manages all data movement	Shared variables; familiar shared-memory model
Code reuse	Must rewrite existing shared-memory code	Can run existing shared-memory programs directly
Performance	Predictable; programmer controls communication	Implicit page-fault overhead; false sharing risk
Fault isolation	Each node independent; failure containment easier	Shared state complicates recovery
Scalability	Better for large clusters	Consistency overhead limits scale

2020 Q6b / 2021 Q3c / 2022 Q4c / 2024 Q4b — False Sharing (1.5–3 marks each)

Question: What is false sharing? How can it be minimized?

What is False Sharing?

False sharing occurs when two or more processes repeatedly write to *different, logically unrelated variables* that happen to reside on the *same memory page*. Each write causes the page to be invalidated at all other processes (under the invalidate protocol) and triggers a page transfer, even though no real data is being shared.

Example: Process A writes variable x and Process B writes variable y . Both x and y are on Page 5. Every time A writes x , it invalidates B's copy of Page 5 (which contains y). B then triggers a page fault to get Page 5 back. This ping-pong continues with no actual data exchange — only wasted network messages.

Why it is costly: The invalidate protocol was designed for real sharing. False sharing causes it to generate the same overhead (multicast + page transfer) for zero logical benefit, severely degrading DSM performance.

How to Minimize

1. **Tune page size.** A page that is too large increases the probability that two unrelated variables share a page. A page that is too small increases the frequency of page transfers on legitimate accesses. Choose a moderate page size matched to each process's working set.
2. **Co-locate related data; separate unrelated data.** Place variables accessed together by the same process on the same page. Place variables written by different processes on *different* pages. This can be done by controlling memory layout in the program (e.g., padding structures to page boundaries, using separate arrays per process).
3. **Use the update protocol for fine-grained sharing.** The update protocol sends only the changed value (not the whole page) and does not invalidate; it is better

when two processes genuinely share small variables within a page. However, for large pages or mostly-exclusive access, invalidate remains preferable.

2022 Q8c — Invalidate vs Update Protocol; Which is Preferable? (3 marks)

Question: Compare the Invalidate and Update protocols in DSM. Which is preferable and why?

	Invalidate	Update
On write	Multicast invalidate to all R-copy holders; only owner holds page (W state)	Multicast new value to all holders; all update in place
Copies	Only one W copy at a time	Multiple W copies (all updated)
Message content	Small (just “invalidate page X”)	Larger (carries new value)
Next read after write	Reader must fetch the page (page fault)	Reader uses local updated copy immediately
Best for	Sequential write patterns; producer-consumer; writes are infrequent or exclusive	Fine-grained sharing; small frequently-written variables; many readers
False sharing	Causes heavy ping-pong	Causes repeated value broadcasts

Which is preferable? Invalidate is the default and generally preferred because:

1. Most programs have *producer-consumer* or *exclusive-write* patterns: one process writes a region, then another reads it. Invalidate handles this with low overhead (one invalidate message per write burst).
2. Invalidate sends only a short control message; the page is only fetched if actually needed. Update always sends the full value even to processes that may never read it again.
3. Sequential consistency is easier to maintain with invalidate (at most one W copy exists at any time).

Update is preferred only when writes are to small variables read immediately by many processes (e.g., a shared counter updated by one and read by all).

2020 Q4c — Why Simple LRU Fails as DSM Block Replacement Policy (2.25 marks)

Question: Why does simple LRU (Least Recently Used) fail as a block replacement policy in DSM?

Answer

In a conventional cache or buffer, LRU is effective: evict the block not accessed for the longest time, because future accesses are likely to recent blocks (temporal locality). DSM

blocks have *state* and *ownership* that LRU ignores:

Reason 1 — Owner blocks require explicit transfer. If a process owns a page in W state (it is the only copy), evicting that page means the data would be lost. Before eviction, the process must: (a) transfer the page and ownership to another process, or (b) write it back to a home node. Simple LRU makes no distinction between “I am the only copy” and “I am one of many copies”. Evicting an owner block costs a network transfer; evicting a non-owner R copy costs nothing (another copy exists). LRU cannot make this distinction.

Reason 2 — Block state affects eviction cost asymmetrically. A W-state (exclusive) block costs far more to evict than an R-state (shared, read-only) block. An LRU eviction candidate might be a W-state block with high eviction cost, when a slightly-more-recent R-state block could be evicted for free. A cost-aware replacement policy (evict the cheapest to evict, breaking ties by recency) is needed.

Reason 3 — Local access frequency $\neq$ global importance. A page rarely accessed locally might be the *only copy* in the DSM (the owner in W state). Evicting it forces ownership transfer. LRU based on local access history is blind to global ownership state.

Correct approach: A DSM-aware replacement policy first evicts non-owner R-state blocks (cheapest: just discard), then non-owner R-state blocks with no other reader, and only lastly owner blocks (requiring ownership transfer protocol).

2020 Q6c — NRNMB / NRMB / RMB / RNMB Replication Strategies (4 marks)

Question: Describe the advantages and disadvantages of the NRNMB, NRMB, RMB, and RNMB strategies in DSM design.

Strategy Overview

The four strategies differ on two axes:

- **Replicated (R) vs Non-Replicated (NR):** whether multiple copies of a block exist.
- **Migrating Block (MB) vs Non-Migrating Block (NMB):** whether the block’s home location changes to follow access patterns.

Strategy	Advantages	Disadvantages
NRNMB		
Non-Replicated Non-Migrating	Simple: single copy at fixed server; no consistency maintenance needed; low memory usage	Server becomes a bottleneck (all accesses go to same node); no fault tolerance; high latency for remote accesses
NRMB		
Non-Replicated Migrating	Block migrates to requesting process; subsequent accesses are local (no network); good for sequential access by one process at a time	Single copy still limits concurrency; migration overhead on each ownership change; bad for multiple readers (only one can own at a time)
RMB		
Replicated Migrating	Multiple copies exist and migrate to requesters; read-heavy workloads benefit (reads from local copy); good if one process reads for a long time	Write invalidates all copies across the network; high invalidation overhead on writes; complex consistency maintenance
RNMB		
Replicated Non-Migrating	Multiple copies at fixed locations; reads always local if copy exists; good for read-mostly data with stable access patterns	Write must update/invalidate all fixed replicas; not adaptive to changing access patterns; higher memory usage; consistency overhead on writes

Summary: NRNMB is simplest but least scalable. NRMB suits single-threaded migration patterns. RMB suits read-heavy workloads with occasional writes. RNMB suits stable read-mostly data where the set of readers is known.

2021 Q3d / 2022 Q4d — Consistency: Definition + Types in DSM (2–3 marks)

Question: What is consistency? What types of consistency can be implemented with DSM?

Definition

Consistency in a distributed/replicated system defines the contract between the system and programmers regarding how writes are propagated and when they become visible across processes. It answers: if process A writes a value and process B reads it, what does B see and when?

Consistency Models Implementable with DSM

1. **Sequential consistency:** The result of any execution is the same as if the operations of all processes were executed in some sequential order, and each process's operations appear in that sequence. All processes agree on the same global order. Achievable with total-order multicast.
2. **Causal consistency:** Only *causally related* writes (writes that have a happen-before relationship) must be seen in causal order everywhere. Concurrent (independent) writes may be seen in different orders by different processes. Uses vector clocks.
3. **Release consistency:** Synchronisation operations are distinguished as *acquire* (lock) and *release* (unlock). All shared-variable accesses before a release are completed before the release. All shared-variable accesses after an acquire see all updates released before the acquire. Between critical sections, no ordering is imposed.
4. **Weak consistency:** Consistency enforced only at *synchronisation points* (explicit barrier or lock operations). Between synchronisation points, processes may see stale data. Weaker and cheaper than release consistency.
5. **Eventual consistency:** Writes will eventually propagate to all replicas; no ordering guarantees in the short term. Used in highly available systems (e.g., DNS, Dynamo).

DSM context: Release consistency is the most common choice for DSM because it is the weakest model that guarantees correct behavior for programs that use standard lock-based synchronisation. It minimises network messages (only flush at release boundaries) while preserving programmer expectations.

2020 Q4a — Causal Consistency + Application (3 marks)

Question: What is causal consistency? Give an example of an application best suited for it.

Definition

Causal consistency guarantees that writes that are *causally related* (one happened-before the other) are seen in the same causal order by *all* processes. Concurrent writes (with no causal relationship) may be seen in different orders at different processes — this is permitted.

Two operations are causally related if: (1) they occur at the same process in order, or (2) one is a write and the other is a read of the same value (read-from relationship), or (3) transitively chained through these.

Implementation: Vector clocks track causal dependencies. A process delays delivering a message until all causally prior messages have been delivered.

Best-Suited Application: Social Media / Discussion Forum

A social media timeline or discussion forum is best suited for causal consistency:

Scenario: User A posts a message m_1 . User B reads m_1 and replies with m_2 (causally after m_1). All users must see m_1 before m_2 , otherwise the reply appears without its context.

With causal consistency:

- $m_1 \rightarrow m_2$ (causal order) is preserved at all replicas: m_2 is not delivered until m_1 is.
- Two independent posts by A and C with no causal relationship may appear in different orders on different users' feeds — this is acceptable.

Strong consistency would be unnecessarily expensive (total order on every post); eventual consistency would allow replies to appear before their original posts. Causal consistency is the sweet spot.

2020 Q4b — Weak vs Release Consistency; Which for DSM? (3.5 marks)

Question: Differentiate weak consistency and release consistency. Which is better suited for DSM design and why?

Weak Consistency

- Shared memory is consistent only at **synchronisation points** (e.g., barriers, explicit sync operations).
- Rule: before a sync operation, all preceding accesses to shared variables must complete. After a sync, all previous writes are visible to all processes.
- No distinction between different types of synchronisation (lock acquire, lock release, barrier are all treated identically as “sync”).

Release Consistency

- Refines weak consistency by distinguishing two synchronisation types:
 - **Acquire:** before acquiring a lock, import all updates that were released by any previous release of the same lock.
 - **Release:** before releasing a lock, all preceding shared-variable writes must be visible (flushed) to other processes.
- Weaker than weak consistency: only propagates updates at release points, not at every sync.

	Weak Consistency	Release Consistency
Sync granularity	Any sync operation flushes all	Only release flushes; acquire imports selectively
Messages generated	More (flush on every sync)	Fewer (flush only on release)
Flexibility	Less (no acquire/release distinction)	More (can pipeline acquires)
Overhead	Higher	Lower

Which is Better for DSM?

Release consistency is better suited for DSM because:

1. **Fewer network messages.** Release consistency only propagates writes at release boundaries. Weak consistency propagates at every sync point, which generates more invalidation/update messages.
2. **Compatible with standard locking.** Programs using mutexes (acquire = lock, release = unlock) work correctly under release consistency without modification.
3. **Lazy release consistency:** An optimisation where updates are sent not at release time but at the next acquire of the same lock — further reducing network overhead. This is specific to release consistency and not possible under weak consistency.

2021 Q2a / 2021 Q2b — Strong vs Weak for Specific Scenarios (3 + 1.75 marks)

Question (Q2a): Differentiate strong and weak consistency. Discuss advantages and disadvantages of each for a video-sharing site.

Question (Q2b): Why is strong consistency required for a law enforcement video evidence server?

Strong vs Weak Consistency

	Strong (Linearizability)	Weak Consistency
Guarantee	Every read returns the most recent write; all nodes see same state instantaneously	Writes propagate eventually; reads may return stale data
Ordering	Total order consistent with real time	No ordering guarantee between operations
Performance	Slow (coordination needed on every read/write)	Fast (local reads from cached/replicated data)
Availability	Lower (blocked during failure/coordination)	Higher (can serve reads even if some replicas down)

For a Video-Sharing Site (e.g., YouTube-style)

Weak consistency is preferable:

- **Advantage:** If a user uploads a video and view counts are slightly stale for a few seconds, it is acceptable. Weak consistency allows serving millions of read requests from local replicas without global coordination, giving high throughput and low latency.
- **Disadvantage:** A user might see an old view count or briefly not see a recently uploaded video. This is tolerable — the inconsistency is minor and self-correcting.

Strong consistency disadvantage for this site: Every read of a view counter would require a global coordination round to ensure the count is current. This would make the system orders of magnitude slower and unable to handle the load.

Why Strong Consistency for Law Enforcement Video Evidence (Q2b)

Strong consistency is **required** for a law enforcement video evidence server because:

1. **Legal integrity:** Any update to evidence (chain of custody change, access log entry, modification flag) must be immediately and identically visible to all authorised parties. If a stale cached read returns an old version after an update (e.g., a tampering event), the inconsistency could conceal that tampering.
2. **Non-repudiation:** Two investigators querying the same evidence file at the same time must see the same version. Under weak consistency, one might see a tampered version and the other the original — creating conflicting records.
3. **Audit requirements:** Legal proceedings require a single, consistent, timestamped history of all accesses and modifications. This requires linearizability so that any read reflects all prior writes in real-time order.

Performance is secondary to correctness for evidence storage. The cost of strong consistency (higher latency) is acceptable.

2021 Q2c — Quorum Read/Write Constraints: 9 Replicas (4 marks)

Question: For a system with 9 replicas, state the constraints on read quorum (R) and write quorum (W) sizes. Give a worked example.

Quorum Constraints

With N replicas, two rules must hold:

1. **Read-Write overlap:** $R + W > N$
Ensures every read quorum shares at least one replica with every write quorum. The shared replica has the latest write, so the reader always gets the current value.
2. **Write-Write overlap:** $W > N/2$
Ensures any two write quorums share at least one replica. This prevents two concurrent writes both succeeding and creating a split-brain.

For $N = 9$

Constraint 1: $R + W > 9$

Constraint 2: $W > 9/2 = 4.5$, so $W \geq 5$

Valid configurations:

W	Min R (from $R > 9 - W$)	$R + W$	Both constraints?
5	5	10	✓
6	4	10	✓
7	3	10	✓
8	2	10	✓
9	1	10	✓

Worked example: Choose $W = 5$, $R = 5$.

- A write succeeds only if acknowledged by 5 of 9 replicas.
- A read contacts 5 replicas and takes the value with the highest version number.

- Since $5 + 5 = 10 > 9$, any read set of 5 replicas must overlap with any write set of 5 by at least 1 ($5 + 5 - 9 = 1$). That shared replica holds the latest write. ✓
- Since $5 > 4.5$, any two write sets of 5 overlap by at least 1. No two concurrent writes can both succeed on disjoint sets. ✓

Trade-off: High W , low R (e.g., $W = 9$, $R = 1$) optimises for fast reads. High R , low W (e.g., $W = 5$, $R = 5$) balances read and write cost. $W = 9$ means all replicas must acknowledge writes (very slow writes, very fast reads).

2024 Q2b — Linearizability + Methods to Ensure Serializability (1.5 marks)

Question: What is linearizability? What methods can be used to ensure serializability?

Linearizability

Linearizability (also called atomic consistency or strong consistency) is the strongest consistency model. It requires that:

1. Every operation appears to take effect **instantaneously** at some point between its invocation and its completion (atomicity).
2. The order of operations is consistent with **real time**: if operation A completes before operation B starts, A appears before B in the global order.

Linearizability implies sequential consistency (both require a consistent global order) but additionally enforces real-time ordering. It is the correctness condition used for distributed locks, atomic registers, and coordination services like ZooKeeper.

Methods to Ensure Serializability

Serializability requires that the outcome of concurrent transactions equals some serial execution of those transactions.

1. **Two-Phase Locking (2PL):** A transaction acquires all locks before releasing any. Growing phase: only acquire. Shrinking phase: only release. Ensures no two transactions hold conflicting locks simultaneously.
2. **Timestamp Ordering:** Each transaction is assigned a unique timestamp on start. Operations proceed only if they are consistent with the timestamp order. If a transaction reads/writes out of order, it is aborted and restarted with a new timestamp.
3. **Optimistic Concurrency Control (OCC):** Transactions execute freely (no locks). At commit time, a validation phase checks whether any conflicting operations occurred. If conflicts detected → abort; otherwise → commit. Best for low-conflict workloads.
4. **Multi-Version Concurrency Control (MVCC):** Multiple versions of each data item are maintained. Reads access the version corresponding to the transaction's start timestamp; writes create new versions. Readers never block writers; good for read-heavy workloads.

Summary Table

Year	Q	Marks	Core answer
2021	Q8b	3.5	DSM = virtual shared pages; cache + page fault + multicast; R/W state table
2022	Q8b	3	Same as above
2024	Q4a	1.5	MP: explicit, scalable; DSM: shared-var model, false sharing risk
2020	Q6a	1.75	Schematic: processes + caches + cloud network + page state labels
2020	Q6b	3	False sharing = unrelated vars on same page $\Rightarrow$ ping-pong; fix: tune page size,
2021	Q3c	2	Same as above
2022	Q4c	2	Same as above
2024	Q4b	1.5	Same as above
2022	Q8c	3	Invalidate: one W copy, small msg; Update: all copies, large msg; Invalidate pr
2020	Q4c	2.25	LRU ignores owner/W-state eviction cost; DSM needs state-aware replacement
2020	Q6c	4	NRNMB (simple, bottleneck) $\cdot$ NRMB (migrates, bad concurrent) $\cdot$ RMB (repl
2021	Q3d	2	Consistency = contract on write visibility; types: sequential, causal, release, we
2022	Q4d	3	Same as above
2020	Q4a	3	Causal = causally-related writes ordered everywhere; social media reply orderin
2020	Q4b	3.5	Release consistency preferred for DSM: fewer msgs, lazy-release possible
2021	Q2a	3	Strong = real-time, slow; Weak = stale, fast; video site $\Rightarrow$ weak
2021	Q2b	1.75	Evidence server $\Rightarrow$ strong: no stale reads, non-repudiation, legal audit
2021	Q2c	4	$R + W > 9$, $W \geq 5$; example $W = 5$, $R = 5$; quorum intersection proof
2024	Q2b	1.5	Linearizability = atomic + real-time order; serializability via 2PL / timestamp
Total		≈ 47	

Five facts to know cold:

1. **False sharing** = unrelated vars on same page $\Rightarrow$ invalidate ping-pong. Fix: separate vars across pages. Asked ALL 4 years.
2. **Invalidate preferred** over update (default for DSM). Know when update is better: fine-grained sharing, many readers.
3. **Release > Weak for DSM**: release distinguishes acquire/release, generates fewer messages, enables lazy release.
4. **Quorum rule**: $R + W > N$ and $W > N/2$. For $N = 9$: $W \geq 5$.
5. **LRU fails** because it ignores page state (owner/W-state blocks cost more to evict than non-owner R copies).